

■ testproject.zip

Source Code Export

■ testproject/

◆ testproject/server.js

```
1 const express = require('express');
2 const mongoose = require('mongoose');
3 const app = express();
4
5 app.use(express.json());
6 app.use('/api/auth', require('./routes/auth'));
7
8 mongoose.connect(process.env.MONGO_URI).then(() => {
9   app.listen(5000, () => console.log('Server running on port 5000'));
10 });
```

■ testproject/controllers/

◆ testproject/controllers/adminController.js

```
1 const User = require('../models/User');
2
3 // @desc Get all users
4 // @route GET /api/admin/users
5 exports.getAllUsers = async (req, res) => {
6   try {
7     const users = await User.find().select('-password');
8     res.status(200).json({ success: true, data: users });
9   } catch (err) {
10    res.status(500).json({ message: err.message });
11   }
12 };
```

◆ testproject/controllers/authController.js

```
1 const jwt = require('jsonwebtoken');
2 const User = require('../models/User');
3
4 exports.login = async (req, res) => {
5   const { email, password } = req.body;
6   const user = await User.findOne({ email });
7   if (!user || !(await user.matchPassword(password))) {
8     return res.status(401).json({ message: 'Invalid credentials' });
9   }
10  const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '7d' });
11  res.json({ token });
12 };
```

■ testproject/middleware/

◆ testproject/middleware/protect.js

```
1 const jwt = require('jsonwebtoken');
2 module.exports = (req, res, next) => {
3   const token = req.headers.authorization?.split(' ')[1];
4   if (!token) return res.status(401).json({ message: 'Not authorised' });
5   try {
6     req.user = jwt.verify(token, process.env.JWT_SECRET);
```

```
7 next();
8 } catch {
9 res.status(401).json({ message: 'Token invalid' });
10 }
11 };
```

■ testproject/models/

◆ testproject/models/User.js

```
1 const mongoose = require('mongoose');
2 const bcrypt = require('bcryptjs');
3
4 const userSchema = new mongoose.Schema({
5   name: { type: String, required: true },
6   email: { type: String, required: true, unique: true },
7   password: { type: String, required: true },
8   role: { type: String, enum: ['admin', 'user'], default: 'user' },
9 }, { timestamps: true });
10
11 userSchema.pre('save', async function () {
12   if (this.isModified('password'))
13     this.password = await bcrypt.hash(this.password, 10);
14 });
15
16 userSchema.methods.matchPassword = function (plain) {
17   return bcrypt.compare(plain, this.password);
18 };
19
20 module.exports = mongoose.model('User', userSchema);
```

■ testproject/routes/

◆ testproject/routes/auth.js

```
1 const express = require('express');
2 const router = express.Router();
3 const { login } = require('../controllers/authController');
4
5 router.post('/login', login);
6 module.exports = router;
```